



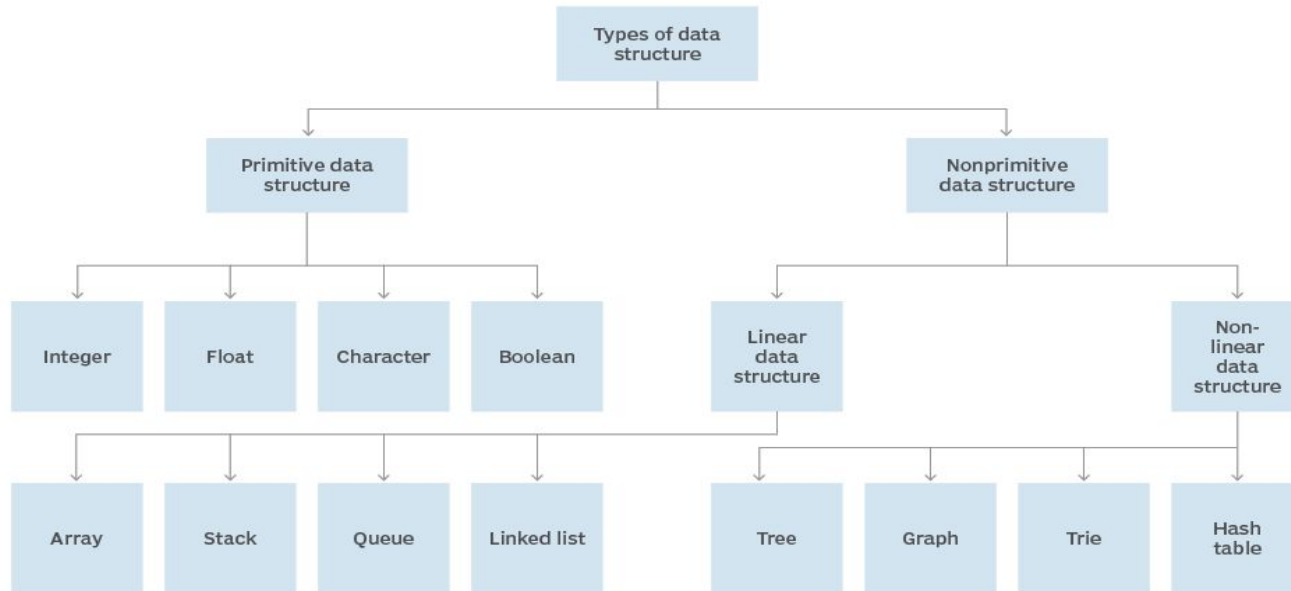
Data Structure

Padmaraj Nidagundi



Last class.....

Data structure hierarchy





Basic Operations on DATA structure

The data in the data structures are processed by certain operations. The particular data structure chosen largely depends on the frequency of the operation that needs to be performed on the data structure.

- **Traversing**
- **Searching**
- **Insertion**
- **Deletion**
- **Sorting**
- **Merging**



Why we need array, we should use variables ?



	0	1	2	3	4	5
Score	0.0	0.0	0.0	0.0	0.0	0.0

Array Storing Floating Values

When a program works with many variables which hold comparable forms of data, then organizational and managerial difficulty quickly arise.

If you are not using arrays, then the number of variables used will increase. Using the array, the number of variables reduces, i.e., you can use a single name for multiple values, you need to deal with its index values (starting from 0 to n).

So if the total run of each player is getting stored in separate variables, using arrays you can bring them all into one array having single name like:
`plrscore[11];`

Operations on Array



Basic Operations

Following are the basic operations supported by an array.

- ▣ **Traverse** – print all the array elements one by one.
- ▣ **Insertion** – Adds an element at the given index.
- ▣ **Deletion** – Deletes an element at the given index.
- ▣ **Search** – Searches an element using the given index or by the value.
- ▣ **Update** – Updates an element at the given index.



How to declare an array?

Here's how you can declare an array in Java:

```
dataType[] arrayName;
```

- `dataType` can be a **primitive data** type like: int, char, Double, byte etc. or an object (will be discussed in later chapters).
- `arrayName` is an **identifier**.

Let's take the above example again.

```
double[] data;
```


Java Array Index

You can access elements of an array by using indices. Let's consider previous example.

```
int[] age = new int[5];
```



Array age of length 5

Student must need to use IDE



Light weight Online Java Compiler - <https://www.codiva.io/>

<https://paiza.io/en/projects/new?language=java>



```
class HelloWorld{
    public static void main(String args[]){
        int array[] = new int[7];
        for (int count=0;count<7;count++){
            array[count]=count+1;
        }
        for (int count=0;count<7;count++){
            System.out.println("array["+count+"] = "+array[count]);
        }
        //System.out.println("Length of Array = "+array.length);
        // array[8] =10;
    }
}
```



STACK - LIFO

Top of stack
(accessible)



Bottom of
stack
(inaccessible)

A **stack** of
four books

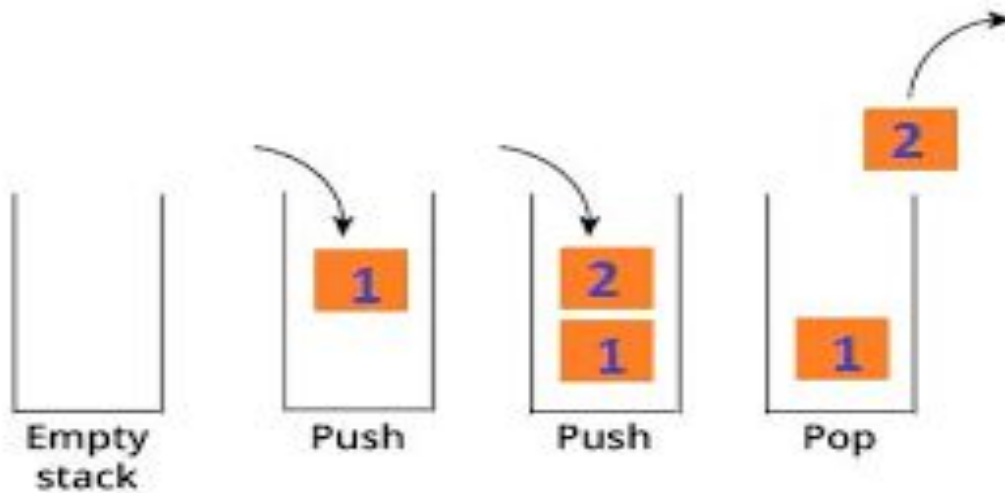


Push a new
book on top



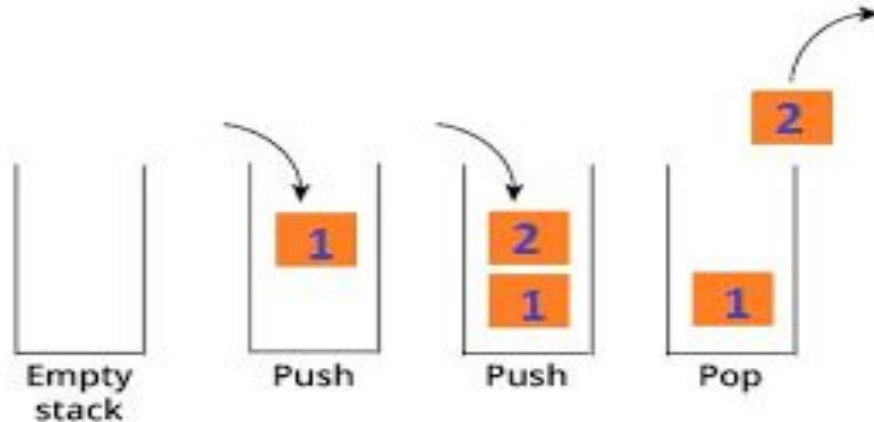
Pop a book
from top

STACK



STACK

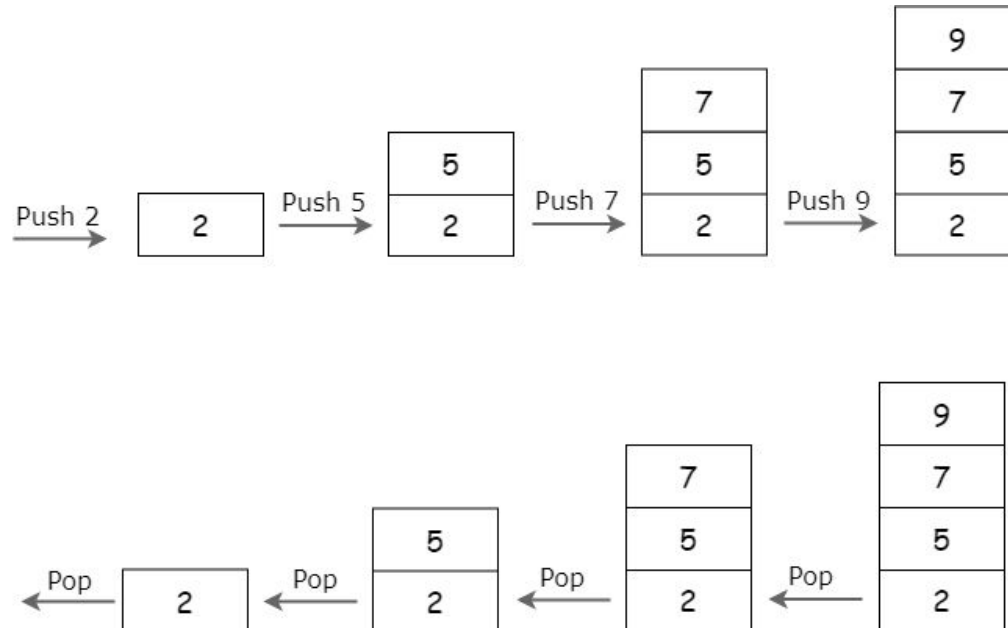
Stack. This collection implements **LIFO**: the last in is the first out. We push elements onto the top of a stack and then pop them for retrieval.



Stack: Array Implementation Visually - Example

<https://www.cs.usfca.edu/~galles/visualization/StackArray.html>

LIFO



Stack Operations



Stack operations may involve initializing the stack, using it and then de-initializing it. Apart from these basic stuffs, a stack is used for the following two primary operations –

- **push()** – Pushing (storing) an element on the stack.
- **pop()** – Removing (accessing) an element from the stack.

When data is PUSHed onto stack.

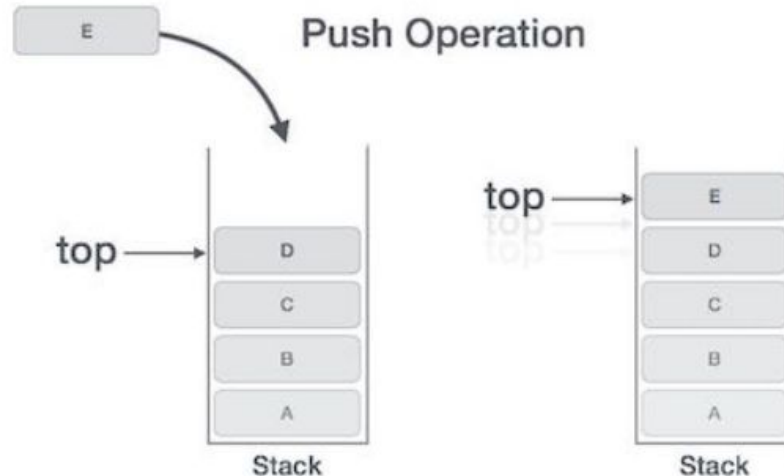
To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks –

- **peek()** – get the top data element of the stack, without removing it.
- **isFull()** – check if stack is full.
- **isEmpty()** – check if stack is empty.

At all times, we maintain a pointer to the last PUSHed data on the stack. As this pointer always represents the top of the stack, hence named **top**. The **top** pointer provides top value of the stack without actually removing it.

Push Operation

- **Step 1** – Checks if the stack is full.
- **Step 2** – If the stack is full, produces an error and exit.
- **Step 3** – If the stack is not full, increments **top** to point next empty space.
- **Step 4** – Adds data element to the stack location, where **top** is pointing.
- **Step 5** – Returns success.



A simple algorithm for Push operation can be derived as follows –

```
begin procedure push: stack, data

    if stack is full
        return null
    endif

    top ← top + 1
    stack[top] ← data

end procedure
```

Implementation of this algorithm in C, is very easy. See the following code –

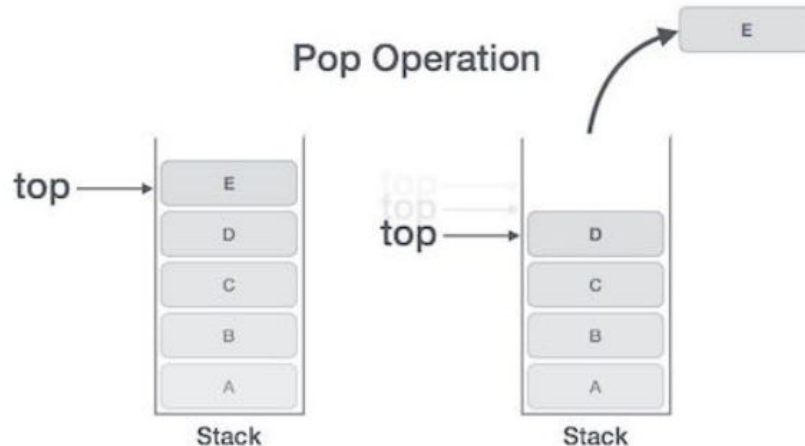
Example

```
void push(int data) {
    if(!isFull()) {
        top = top + 1;
        stack[top] = data;
    } else {
        printf("Could not insert data, Stack is full.\n");
    }
}
```

Pop Operation

A Pop operation may involve the following steps –

- **Step 1** – Checks if the stack is empty.
- **Step 2** – If the stack is empty, produces an error and exit.
- **Step 3** – If the stack is not empty, accesses the data element at which **top** is pointing.
- **Step 4** – Decreases the value of top by 1.
- **Step 5** – Returns success.



A simple algorithm for Pop operation can be derived as follows –

```
begin procedure pop: stack

    if stack is empty
        return null
    endif

    data ← stack[top]
    top ← top - 1
    return data

end procedure
```

Implementation of this algorithm in C, is as follows –

Example

```
int pop(int data) {

    if(!isempty()) {
        data = stack[top];
        top = top - 1;
        return data;
    } else {
        printf("Could not retrieve data, Stack is empty.\n");
    }
}
```



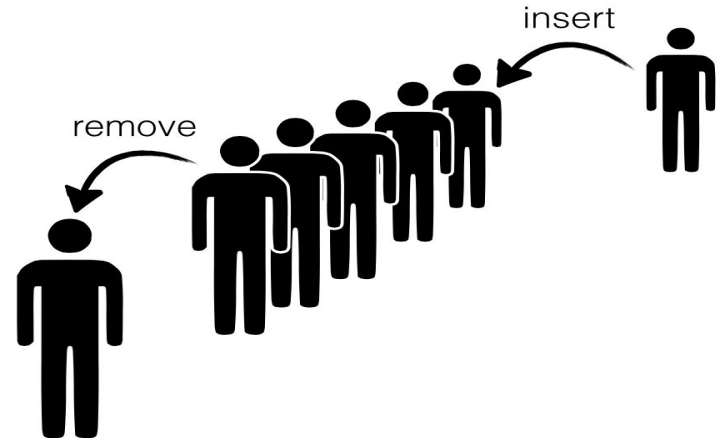
Home task

Write 5 different programs using Stack. Goal of task is to know more about Stack usage in the data structure programming.

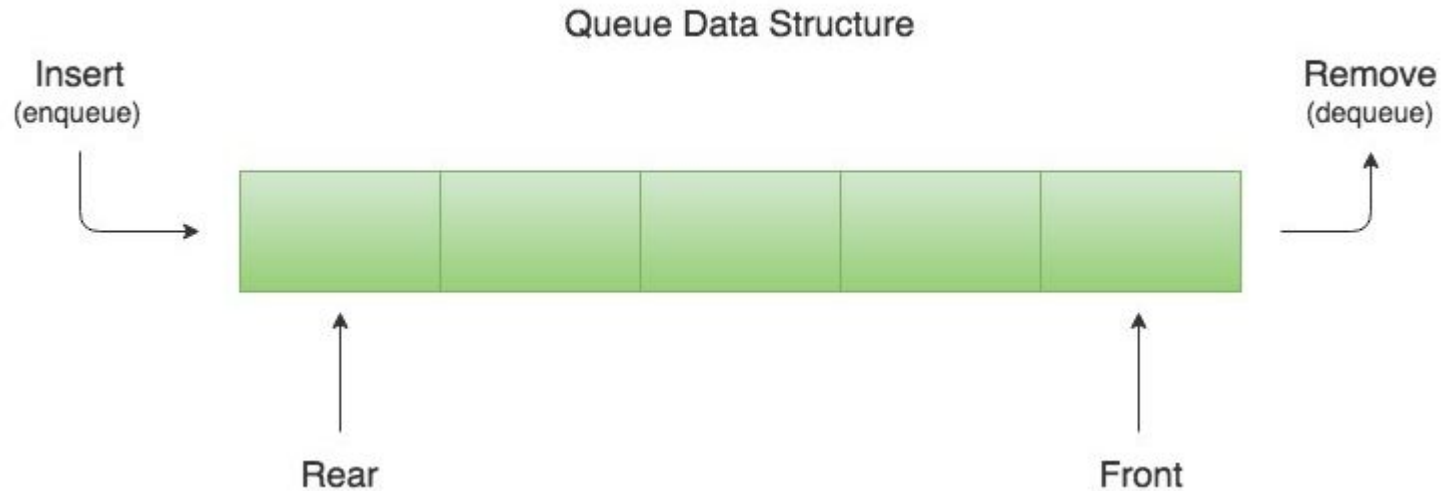


Queue - FIFO

Queue Representation



A Queue is a First In First Out (FIFO) data structure. It models a queue in real-life. Yes, the one that you might have seen in front of a movie theater, a shopping mall, a metro, or a bus.





Operations on Queue

add()- This method is used to add elements at the tail of queue. More specifically, at the last of linked-list if it is used, or according to the priority in case of priority queue implementation.

peek()- This method is used to view the head of queue without removing it. It returns Null if the queue is empty.

element()- This method is similar to peek(). It throws *NoSuchElementException* when the queue is empty.

remove()- This method removes and returns the head of the queue. It throws *NoSuchElementException* when the queue is empty.

poll()- This method removes and returns the head of the queue. It returns null if the queue is empty.

size()- This method return the no. of elements in the queue.

OPERATION	THROWS EXCEPTION	RETURN VALLUES
Insert	add(element)	offer(element)
Remove	remove()	poll()
Examine	element()	peek()

STACK VERSUS QUEUE

Stack	Queue
Stack is an abstract data type represented by a physical stack of entities where insertion and deletion takes place at the same end.	Queue is also an abstract data type similar to stacks except it is open at both the ends.
It is based on the working principle of LIFO meaning last-in-first-out.	It is based on the working principle of FIFO meaning first-in-first-out.
Object can be added in the stack one at a time meaning the object can be removed which is added last.	The object which is added first can only be removed from the queue.
Push adds items to the stack and pop removes recently added items from the stack.	Enqueue adds items to the rear and dequeue removes items from the front of the queue.
Only one pointer is used in a stack.	Two pointers are used in a simple queue case.



Home task

Write 5 different programs using Queue.
Goal of task is to know more about Queue
usage in the data structure programming.



Next class

Queue - programs

Stack - programs

Linked List Introduction

**Gets the element at the front of the queue
without removing it.**